EXPERIMENT NO. 6 C (Group A) Aim: Build a Tic-Tac-Toe game using reinforcement learning in Python by using following tasks a. Setting up the environment b. Defining the Tic-Tac-Toe game c. Building the reinforcement learning model d. Training the model e. Testing the model

Name:Aditya Rade Roll.No:BEAD23167

Outputs:

Step 1 — Import NumPy

In [1]:
```
import numpy as np
```

Step 2 — Define the Tic-Tac-Toe Environment

In [3]:
```
class TicTacToeEnvironment:

    def __init__(self):
        self.state = [0] * 9
        self.is_terminal = False

    def reset(self):
        self.state = [0] * 9
        self.is_terminal = False

    def get_available_moves(self):
        return [
            i for i, mark in enumerate(self.state)
            if mark == 0
        ]

    def make_move(self, move, player_mark):
        self.state[move] = player_mark

    def check_win(self, player_mark):
        winning_states = [
            [0, 1, 2],  # Row 1
            [3, 4, 5],  # Row 2
            [6, 7, 8],  # Row 3
            [0, 3, 6],  # Column 1
            [1, 4, 7],  # Column 2
            [2, 5, 8],  # Column 3
            [0, 4, 8],  # Diagonal
            [2, 4, 6]   # Diagonal
        ]

        for state_indices in winning_states:
            if all(
                self.state[i] == player_mark
                for i in state_indices
            ):
                self.is_terminal = True
```

```
                return True

        return False

    def is_draw(self):
        return 0 not in self.state
```

Step 3 — Test the Environment

In [4]:
```
environment = TicTacToeEnvironment()

print("Initial state:")
print(environment.state)

print("\nAvailable moves:")
print(environment.get_available_moves())
```

```
Initial state:
[0, 0, 0, 0, 0, 0, 0, 0, 0]

Available moves:
[0, 1, 2, 3, 4, 5, 6, 7, 8]
```

Step 4 — Test Making a Move

In [5]:
```
environment.make_move(0, 1)

print("State after Player 1 move:")
print(environment.state)

print("Available moves:")
print(environment.get_available_moves())
```

```
State after Player 1 move:
[1, 0, 0, 0, 0, 0, 0, 0, 0]
Available moves:
[1, 2, 3, 4, 5, 6, 7, 8]
```

Step 5 — Test Win Detection

In [6]:
```
environment.reset()

environment.make_move(0, 1)
environment.make_move(1, 1)
environment.make_move(2, 1)

print("Board:")
print(environment.state)

print("Player 1 won:", environment.check_win(1))
```

```
Board:
[1, 1, 1, 0, 0, 0, 0, 0, 0]
Player 1 won: True
```

Step 6 — Define the Q-Learning Agent

In [7]:
```
class QLearningAgent:

    def __init__(
        self,
        learning_rate=0.9,
        discount_factor=0.9,
        exploration_rate=0.3
    ):
        self.learning_rate = learning_rate
        self.discount_factor = discount_factor
        self.exploration_rate = exploration_rate

        self.q_table = np.zeros((3**9, 9))

    def get_state_index(self, state):

        state_index = 0

        for i, mark in enumerate(state):
            state_index += (3 ** i) * (mark + 1)

        return state_index

    def choose_action(self, state, available_moves):

        state_index = self.get_state_index(state)

        # Exploration
        if np.random.random() < self.exploration_rate:
            return np.random.choice(available_moves)

        # Exploitation
        q_values = self.q_table[
            state_index,
            available_moves
        ]

        return available_moves[
            np.argmax(q_values)
        ]

    def update_q_table(
        self,
        state,
        action,
        next_state,
        reward
    ):

        state_index = self.get_state_index(state)

        if next_state is not None:
            next_state_index = self.get_state_index(next_state)
```

```
            max_q_value = np.max(
                self.q_table[next_state_index]
            )
        else:
            max_q_value = 0

        self.q_table[state_index, action] = (
            (1 - self.learning_rate)
            * self.q_table[state_index, action]
            +
            self.learning_rate
            * (
                reward
                +
                self.discount_factor
                * max_q_value
            )
        )
```

Step 7 — Understand the Q-Learning Parameters

The agent uses:

Learning rate = 0.9 Discount factor = 0.9 Exploration rate = 0.3 Learning rate

Controls how strongly new information changes the existing Q-value.

Discount factor

Controls how much future rewards matter.

Exploration rate

Controls how often the agent tries a random available move instead of using its current best-known move.

Step 8 — Create the Agents

In [8]:
```
agent1 = QLearningAgent()
agent2 = QLearningAgent()

print("Agent 1 created")
print("Agent 2 created")
```

```
Agent 1 created
Agent 2 created
```

Step 9 — Check the Q-Table

In [9]:
```
print("Q-table shape:")
print(agent1.q_table.shape)
```

```
Q-table shape:
(19683, 9)
```

Step 10 — Define the Evaluation/Training Function

In [10]:

```
def evaluate_agents(agent1, agent2, num_episodes=1000):

    environment = TicTacToeEnvironment()

    agent1_wins = 0
    agent2_wins = 0
    draws = 0

    for _ in range(num_episodes):

        environment.reset()

        current_agent = agent1

        while not environment.is_terminal:

            available_moves = environment.get_available_moves()

            current_state = environment.state.copy()

            action = current_agent.choose_action(
                current_state,
                available_moves
            )

            player_mark = (
                1 if current_agent == agent1 else -1
            )

            environment.make_move(
                action,
                player_mark
            )

            # Check whether the current player won
            if environment.check_win(player_mark):

                current_agent.update_q_table(
                    current_state,
                    action,
                    None,
                    10
                )

                if current_agent == agent1:
                    agent1_wins += 1
                else:
                    agent2_wins += 1

                break

            # Check for draw
            elif environment.is_draw():
```

```
                current_agent.update_q_table(
                    current_state,
                    action,
                    None,
                    0
                )

                draws += 1

                break

            # Continue to next state
            next_state = environment.state.copy()

            reward = 0

            current_agent.update_q_table(
                current_state,
                action,
                next_state,
                reward
            )

            # Switch player
            current_agent = (
                agent2
                if current_agent == agent1
                else agent1
            )

    return agent1_wins, agent2_wins, draws
```

Step 11 — Run the Training/Evaluation

In [11]:
```
agent1_wins, agent2_wins, draws = evaluate_agents(
    agent1,
    agent2,
    num_episodes=1000
)
```

Step 12 — Display the Results

In [13]:
```
print(f"Agent 1 wins: {agent1_wins}")
print(f"Agent 2 wins: {agent2_wins}")
print(f"Draws: {draws}")

print("\nTotal games:")
print(agent1_wins + agent2_wins + draws)
print("Name:Aditya Rade Roll No:BEAD23167")
```

```
Agent 1 wins: 675
Agent 2 wins: 243
Draws: 82

Total games:
1000
Name:Aditya Rade Roll No:BEAD23167
```

Step 13 — Optional: Display the Board

In [14]:
```python
def display_board(state):

    symbols = {
        0: " ",
        1: "X",
        -1: "O"
    }

    print(
        f" {symbols[state[0]]} | {symbols[state[1]]} | {symbols[state[2]]} "
    )
    print("---+---+---")
    print(
        f" {symbols[state[3]]} | {symbols[state[4]]} | {symbols[state[5]]} "
    )
    print("---+---+---")
    print(
        f" {symbols[state[6]]} | {symbols[state[7]]} | {symbols[state[8]]} "
    )
```

In [16]:
```python
test_board = [1, -1, 0,
              1, 0, -1,
              0, 0, 1]

display_board(test_board)
print("Name:Aditya Rade Roll No:BEAD23167")
```

```
 X | O |
---+---+---
 X |   | O
---+---+---
   |   | X
Name:Aditya Rade Roll No:BEAD23167
```